ICT103 Programming

Topic 7

Classes and Objects

CIHE Australia

Resources

Prescribed Texts

- McMullen, K., Matthews, E., & Parsons, J. J. (2022). Programming with Python (1st ed.)., Cengage Learning

Recommended Readings

- Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media, Inc

Lesson Learned from Week 6:

- Differentiate between built-in functions, imported functions, and programmer-defined functions.
- State the purpose of a function's return value.
- Explain the meaning of "pass by value" and "pass by reference" in the context of functions

- Describe module structures
- Explain how the keyword import copies code from another module.
- Explain how to create modules
- Understand the purpose of the modules in Python

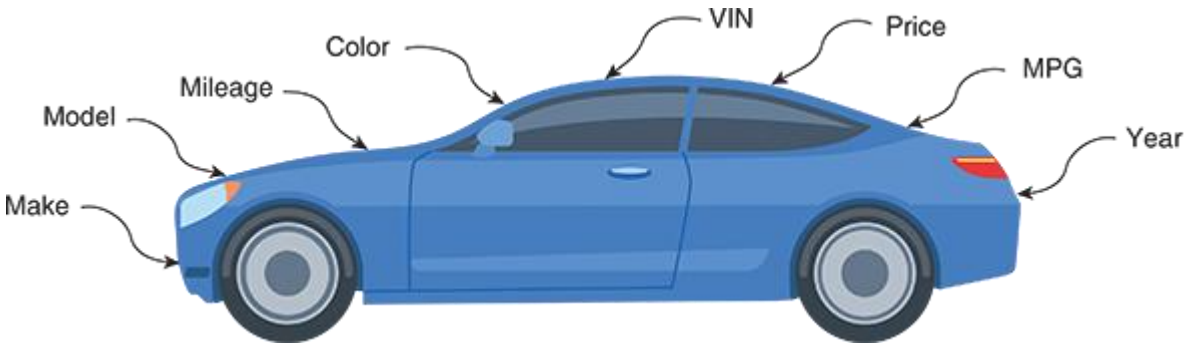
Week 7: Lesson Learning Outcomes

- Understand the difference between object-oriented programming (OOP) and procedural programming
- Exploring concepts of objects and classes in OOP
- Understanding the use of static elements in a Class
- Characteristics of Objects in OOP

Classes in Object-Oriented Programming

- In Procedural programming, code is written step by step to create a set of instructions for the computer to follow

- OOP is more appropriate when your program needs to model a complex, real-world system with many components, actions and relationships.
  - Ex. Banking application



- Ex. Procedural program to manage cars at a dealership
  - store all information in variables
  - As number of changes increased, so would the difficulty of changing code

Classes and Objects

- Python is an object-oriented programming language
- An object is a collection of data (variables) and methods (functions).
- A class is a blueprint for creating that object

Classes

- A class is considered a blueprint of objects
  - Same as architectural blueprint describes a building

```
class Car:
    pass
print(Car)
```

- Example:
  - Class as a sketch (prototype) of a house with details about floors, doors, windows, etc.
  - Build the house; house is the object
  - Since many houses can be made from the same description, many objects can be created from a class

Output

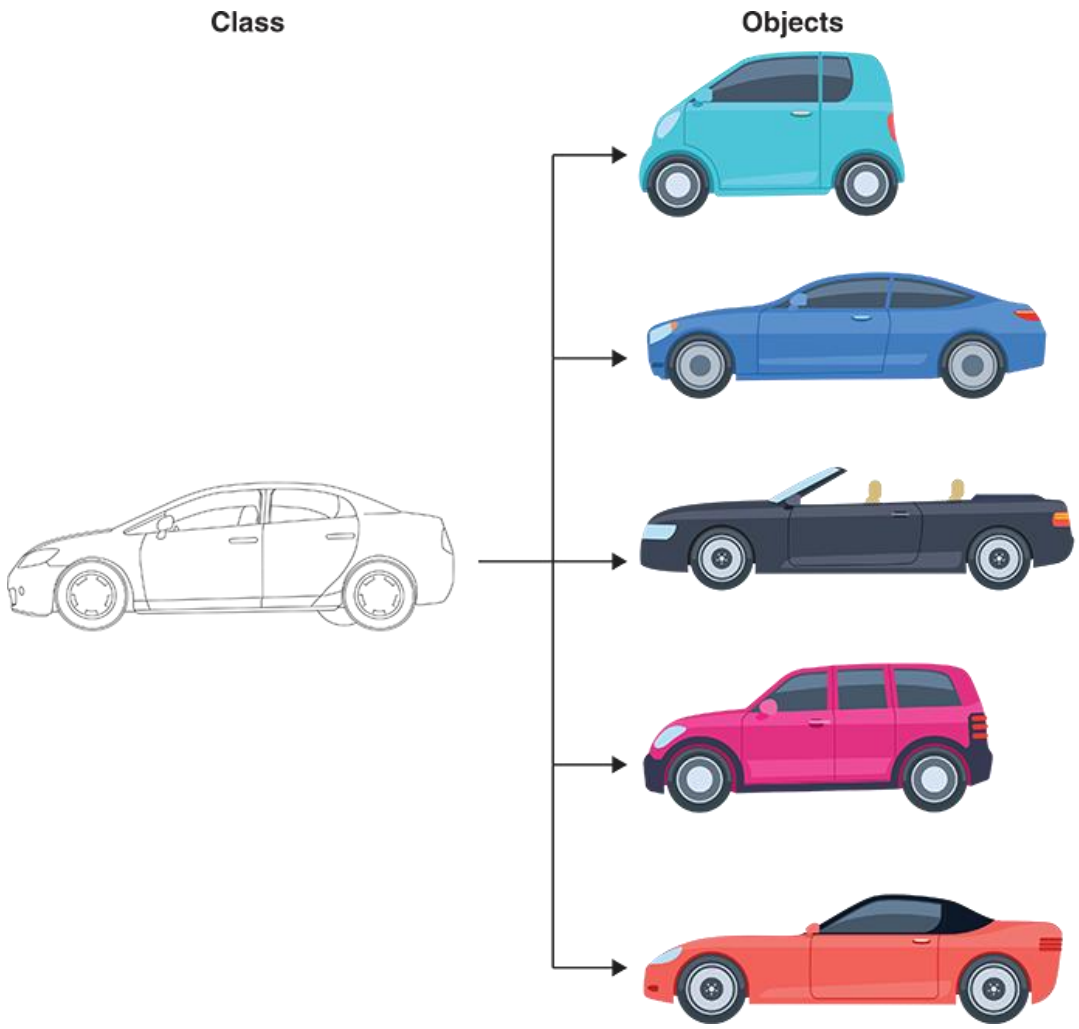
```
<class '__main__.Car'>

=== Code Execution Successful ===
```

Objects

- Each object is an instance of a class.
  - A realized physical entity, while a class is a logical entity
  - A class is defined once, but an object can be created (instantiated) from that class many times

- The code creates a new object from the class Car

```
my_car=Car()
```

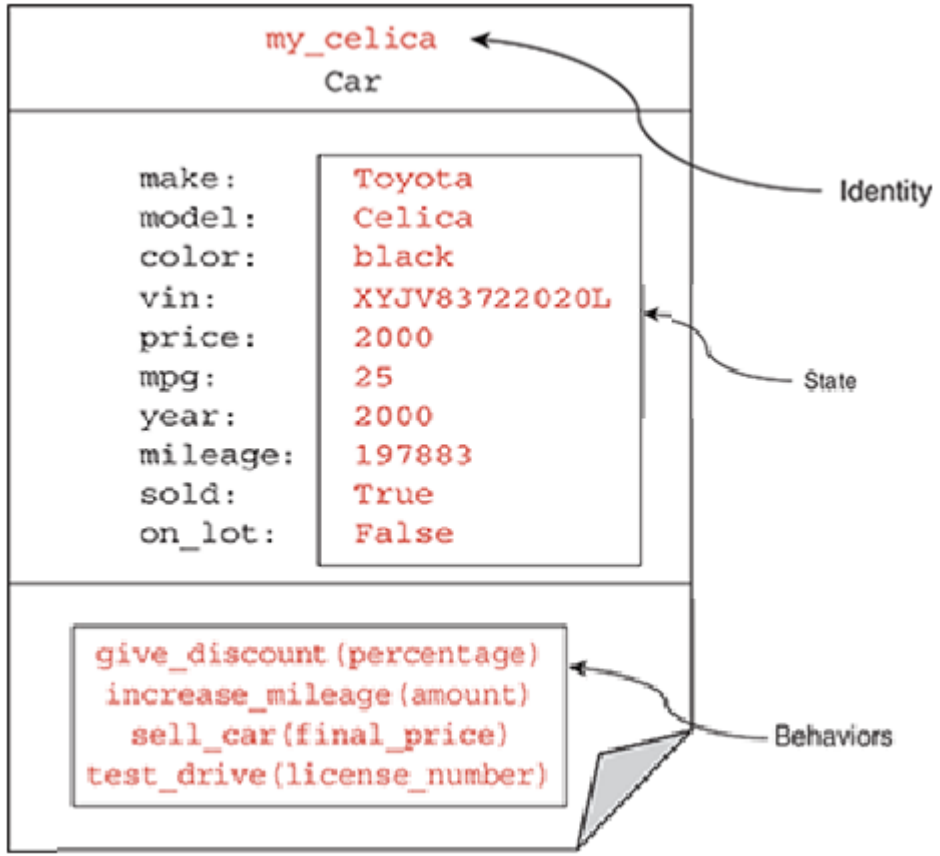


Characteristics of Objects in OOP

- **Object Identity:**
  - Every object must be uniquely identified.

- **Object State:**
  - An object has an attribute that represents a state of an object, and it also reflects the property of an object.

- **Object Behavior:**
  - An object has methods that represent its behavior.



Objects as Variables

- Primitive data types are built-in data types that hold a single value (e.g., integer, decimal, character, Boolean value)

- Objects are complex data types, which can store more than one type of primitive data type

- Ex. Variable name **my_celica** refers to a specific car, the one you created from the **Car** class

```
from vehicle import Car

my_corolla = Car()
my_beetle = Car()
my_lamborghini = Car()

my_corolla.color = "Blue"
my_beetle.model = "Beetle"
my_lamborghini.mileage = 1400
```

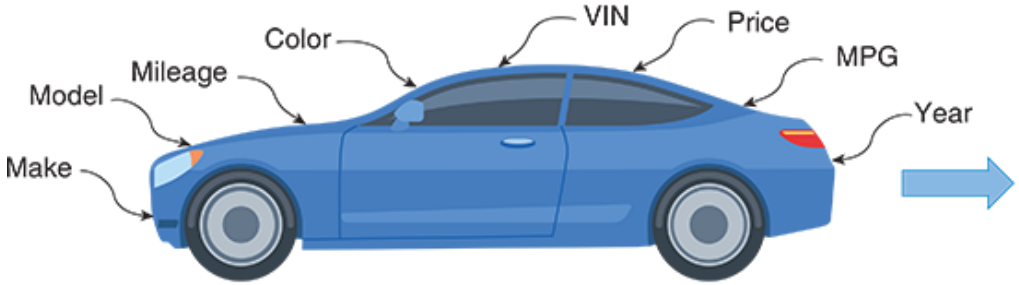
- The keyword **import** is used to import the class from another module named **vehicle**

Class Components

- **class** is a keyword that is used only to define a class
- *Class name*:
  - Identifies the class by a specific name, usually a noun. Ex. **Car**
- *Initialization method (a.k.a **constructor**):*
  - A method containing the starting instructions on how to set up the object.
- *Member variables (a.k.a. attributes or properties)*:
  - Variables that store the data for a class
- *Methods*:
  - Functions that belong to a class and that describe the actions an object can perform, usually named with a verb phrase. Ex. **test_drive()**

Example

```python
class Car():
    def __init__(self):
        self.make = ""
        self.model = ""
        self.color = ""
        self.vin = ""
        self.price = 0
        self.mpg = 0
        self.year = 0
        self.mileage = 0
        self.sold = False
        self.on_lot = False
    def give_discount(self, percentage):
        pass
    def increase_mileage(self, amount):
        pass
    def sell_car(self, final_price):
        pass
    def test_drive(self, license_number):
        pass
```




```
Car
make:      string
model:     string
color:     string
vin:       string
price:     integer
mpg:       integer
year:      integer
mileage:   integer
sold:      boolean
on_lot:    boolean

give_discount(percentage)
increase_mileage(amount)
sell_car(final_price)
test_drive(license_number)
```

MicroOne/Shutterstock.com

__init__ Method

- Known as the constructor
- Special method in Python that sets up a new object
- Automatically called when an object of a class is created
- Allowing class to initialize its attributes and set up object's initial state

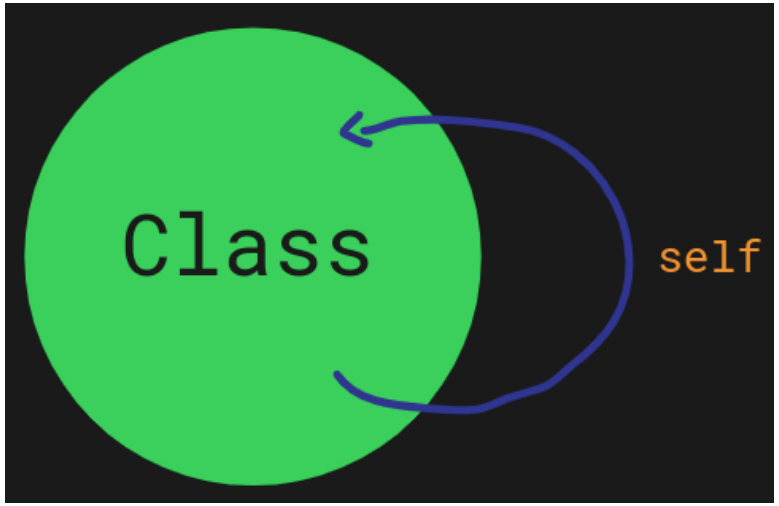
```python
class Car:
    def __init__(self, make, model, year, color):
        self.make = make
        self.model = model
        self.year = year
        self.color = color  # New attribute for color

    def display_info(self):
        return self.year, self.color, self.make, self.model

# Creating an object of Car class with color
my_car = Car("Toyota", "Camry", 2020, "Blue")
print(my_car.display_info())  # Outputs: 2020 Blue Toyota Camry
```

Output

```
(2020, 'Blue', 'Toyota', 'Camry')

=== Code Execution Successful ===
```

What is `self` and why do we use it?

- **`self`** is a reference to the instance of the class that is being created or manipulated
- Allows you to access attributes and methods of the class that differentiates between instance attributes and local variables
- By convention, **`self`** is used as the first parameter in instance methods.




```
class human():
    def __init__(self, age=0, sex="?"):
        self.age = age
        self.sex = sex
    def speak(self):
        print("Hello i am :", self.age, "and ", self.sex)
```

```
man = human()
```

```
woman = human()
```

Attributes (Variables) and Methods

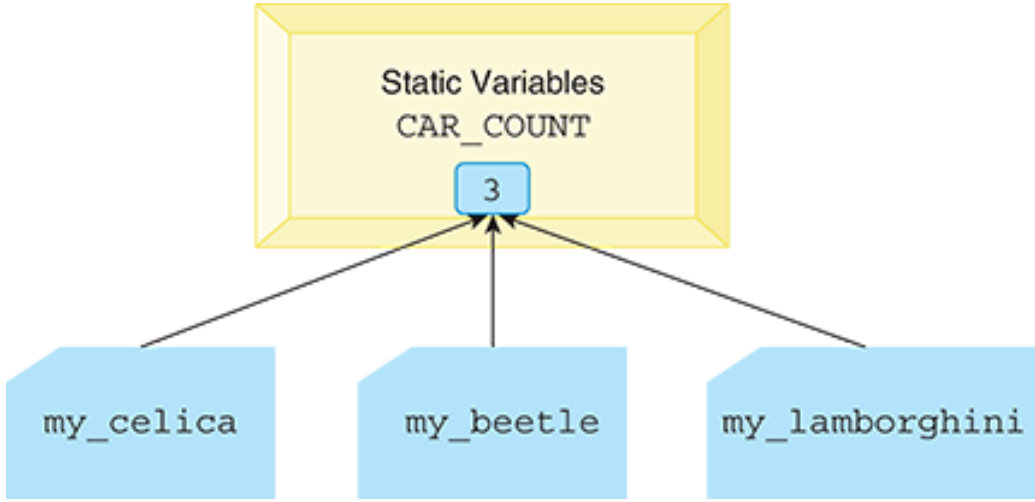
- **Class Variables** are accessible by all objects.
- **Instance Attributes (Variables)** represents the states or properties of an object, like its color, model, or year.
  - Unique to each object
- **Instance Methods** define behaviors or actions that the object can perform, such as starting, stopping, or displaying details.

Car States	Car Object	Car Behaviors
• Color • Current speed • Fuel level		• Starting the engine • Accelerating • Braking

Object-Oriented Features and Principles

- **Encapsulation**: Bundling data with actions it can perform, which makes objects easier to use
  - A class encapsulates or encloses its attributes and methods into an object
  - Enhances coding structures by emphasizing the code's member variables and available actions
  - Provides control over how each object should function
  - Assists with information hiding
- **Information hiding**: concealing an object's data and methods from outside sources
  - An object can access its own data and methods, but other entities cannot
  - Allows programmers to change how a class works without having to modify code that uses objects instantiated from it

Using Static Elements in a Class



- Static variable: a member variable of a class that contains the same value in all instances of the class
- To create a static member of a class, declare the variables outside any method typically before the __init__() method
  - Naming convention is to use all capital letters, ex. CAR_COUNT=0

- Consider using a static variable when the variable's value is independent of the objects and needs to be shared across all objects

Static Methods

- (a.k.a class method): a method that accesses/modifies a static variable, is independent of any particular object of the class, and can be called without creating an instance of the object (because it is called on the class itself)

- To make a method static:
  - Change the first parameter from keyword `self` to a parameter `cls`
  - Add a line containing the `@classmethod` tag just before the method definition

```
class Car():

    CAR_COUNT = 0

    @classmethod
    def increase_car_count(cls):
        cls.CAR_COUNT += 1
    def __init__(self):

        self.make = ""

        self.model = ""

        self.color = ""

        self.vin = ""

        self.price = 0

        self.mpg = 0

        self.year = 0

        self.mileage = 0

        self.sold = False

        self.on_lot = False

    def give_discount(self, percentage):
        pass

    def increase_mileage(self, amount):
        pass

    def sell_car(self, final_price):
        pass

    def test_drive(self, license_number):
        pass
```

New static method

Step-by-step Instructions

- Define the Class
- Add Instance Methods
- Create an Instance
- Call the Methods

```python
class Car:
    def __init__(self, make, model, year):
        self.make = make          # Instance attribute
        self.model = model        # Instance attribute
        self.year = year          # Instance attribute

    def start_engine(self):
        status = "Engine started."  # Local variable
        print(status)               # Prints the engine status

    def display_info(self):
        # Print car information using instance attributes
        print(f"My car is a {self.year} {self.make} {self.model}.")


# Creating an instance of Car
my_car = Car(make="Toyota", model="Corolla", year=2022)

# Calling methods
my_car.start_engine()       # Output: Engine started.
my_car.display_info()       # Output: My car is a 2022 Toyota Corolla.
```

Next Week – What's Coming Up?

- Understand methods and their purpose
- Explain built-in overriding
- Use of Constructors in OOP
- Identify visibility of variables in OOP

CIHE

CIHE Australia